SECURE SOFTWARE ENGINEERING
MECR1053
SECTION 01

LECTURER

TS DR. MOHD ZAMRI BIN OSMAN

FINAL PROJECT

Hands-On Comparative Analysis of Open-Source Security Testing Tools
A SAST and DAST Evaluation on Intentionally Vulnerable Applications

NAME	KEERTENNAH DEVI A/P PONNAMBALAM
MATRIC NO.	MEC254026
VIDEO LINK	https://drive.google.com/file/d/15PMHBhxTXvH7vCj0nDrLUs4BafOFIVN_/view?usp=sharing

1.0 Application Overview and Tool Selection Justification

1.1 Target Application Profile

OWASP Juice Shop (v16.0.0) has been chosen for this assessment. It is an intentionally vulnerable web application created by the Open Web Application Security Project (OWASP). This application is a modern Node.js/Express/Angular application designed for the purpose of security training with the ability to practice penetration testing. It has over 100 documentable vulnerabilities, including SQL Injection attacks, Cross-Site Scripting (XSS) attacks, insecure authentication, broken access control and cryptographic failures.

The application is deployed on my local workstation using Docker within a fully isolated container environment. This type of deployment provides a complete level of reproducibility, eliminates risks associated with network exposure and allows me to instantly tear the application down and rebuild it. The container was mapped to the host machine's loopback interface to simulate a local development environment and prevent any external traffic.

1.2 Tool Selection Justification

I chose three different open source tools to fulfill the multidimensional testing requirements of the Secure Software Development Life Cycle (SSDLC). It fit into three distinct paradigms for security testing:

- Semgrep (SAST): Semgrep is a lightweight, fast static analysis tool that uses source code to find dangerous syntax patterns, hard-coded secrets and structural vulnerabilities before code is deployed into production.Since Semgrep is compatible with Node.js and TypeScript, it is well suited to test and evaluate Juice Shop's code base.
- OWASP ZAP (DAST): OWASP ZAP is the industry standard for dynamic testing that analyzes a running container from the perspective of an external attacker. ZAP identifies a running application's exposure through HTTP headers, such as session management and cookies, along with application

behaviour, something that a SAST tool will not necessarily find by looking at the source code alone.

- Trivy (SCA/Secret Detection): Trivy was integrated into our testing plan as the third tool to scan for exposed hard-coded credentials, active cryptographic keys, and configuration mistakes in the code base. Trivy helps fill the gap where a pure syntactic SAST tool may miss structural text exposures.

2.0 Environment Setup and Test Execution Methodology

2.1 Testing Environment

Window command prompt. Docker was used to containerize the target application, allowing it to be deployed and torn down in a fully reproducible and controlled manner.

Table 1: Testing environment

Component	Description
Host Operating System	Microsoft Windows 11
Containerization Platform	Docker Desktop (v24.0+)
Target Application	OWASP Juice Shop (v16.0.0)
Deployment Method	Docker Container
Access URL	http://localhost:8080
Testing Isolation	Fully local (no external network exposure)

2.2 Target Deployment

The official Docker image was used to deploy the Juice Shop container. The initial deployment encountered an issue where Port 3000 was blocked by another local service running on the host system, necessitating a change in the dynamic port mapping from 8080 to 3000 for accessibility purposes and without making any changes to the host firewall rules.

```
C:\Users\User>docker rm -f juice-shop
juice-shop

C:\Users\User>docker run -d --name juice-shop -p 8080:3000 bkimminich/juice-shop:v16.0.0
d3dc2cfbfd34b784848bf6718b295b92c1b0d4214e0d6f32097cadf8f2e8357a
```

FIGURE 1: Docker run command showing port 8080:3000 mapping

2.3 Tool 1: Semgrep (SAST)

Docker used to run Semgrep locally against a workspace directory mounted through the Docker interface. The core local fallback p/security-audit ruleset was successfully executed to bypass the registry constraint.

FIGURE 2: Semgrep execution showing 1,003 files scanned

2.4 Tool 2: DAST (OWASP ZAP)

The stable OWASP ZAP container was downloaded from GitHub Container Registry (ghcr.io) and executed using the zap-baseline.py automation framework. The dynamic container border was scanned through the loopback interface of the Docker host.

FIGURE 3: ZAP baseline scan execution

2.5 Tool 3: SCA/Secret Scan (Trivy)

Trivy was executed on the local file system mount to look for hidden secrets and embedded cryptographic key pairs within source files.

FIGURE 4: Trivy filesystem scan execution

3.0 Raw Tool Findings & Analysis

3.1 SAST (Semgrep) Analysis Summary

- Total Targets Scanned: 1,003 files (tracked by git)
- Total Rules Applied: 225 core rules (across TS, JS, JSON, and Dockerfile layout)
- Total Findings Identified: 7 Critical/Blocking Findings

Table 2: Semgrep Detected Findings

Target File	Rule Reference & Vulnerability Type	Severity	Code Context / Fragment
frontend/.../navbar.component.html	unquoted-attribute-var (XSS Risk)	Blocking	<img [src]="logoSrc" class="logo" alt={{applicationName}}>
frontend/.../purchase-basket...html	unquoted-attribute-var (XSS Risk)	Blocking	<img [src]="'...' + element.image" alt={{element.name}}
lib/insecurity.ts	jwt-hardcode (Hardcoded Secret)	Blocking	jwt.sign(user, privateKey, { expiresIn: '6h', algorithm: 'RS256' })
routes/redirect.ts	unknown-value-in-redirect (Open Redirect)	Blocking	res.redirect(toUrl)
routes/videoHandler.ts	unknown-value-with-script-tag (Stored/Reflected XSS)	Blocking	compiledTemplate.replace('<script id="subtitle"></script>', '... type="text/vtt" ...>' + subs + '</script>')
views/dataErasureForm.hbs	unquoted-attribute-var (Template Injection/XSS)	Blocking	<input type="email" required placeholder={{userEmail}} name="email"...>

3.2 DAST (OWASP ZAP) Analysis Summary

The ZAP baseline scan processed the live application interfaces successfully, issuing 9 distinct alert warnings across 58 total pass indicators.

Table 3: OWASP ZAP Alert Classifications

Alert Identifier	Rule Name & Vulnerability Classification	Instance Count	Risk Impact / Status
10017	Cross-Domain JavaScript Source File Inclusion	5	Warning (New)
10038	Content Security Policy (CSP) Header Not Set	5	Warning (New)
10049	Non-Storable Content	10	Warning (New)
10063	Deprecated Feature Policy Header Set	5	Warning (New)
10096	Timestamp Disclosure - Unix	1	Warning (New)
10098	Cross-Domain Misconfiguration	5	Warning (New)
10109	Modern Web Application	5	Information/Warning
10110	Dangerous JS Functions	2	Warning (New)
90004	Cross-Origin-Embedder-Policy Header Missing/Invalid	10	Warning (New)

3.3 SCA & Secrets Discovery (Trivy) Summary

Trivy parsed the mounted application structures isolating **3** highly critical secrets exposed in plain text within test suites and helper libraries.

- Finding 1 (Medium Severity): frontend/src/app/app.guard.spec.ts (Line 46)
    - Hardcoded JSON Web Token (JWT) string injected directly inside local storage configurations for testing specs.
- Finding 2 (Medium Severity): frontend/src/app/last-login-ip/last-login ip.component.spec.ts (Line 72)
    - Insecure active mockup JWT session token written explicitly inside client unit test variables.
- Finding 3 (High Severity): lib/insecurity.ts (Line 21)
    - Exposed Asymmetric Private Key (-----BEGIN RSA PRIVATE KEY-----). Hardcoded core asymmetric private key material used to sign identity assertions.

4.0Triangulation and Verification Analysis

4.1 Sample Manual Verification (True vs. False Positives)

The following tables present manual validation of the most critical findings.

Table 4: Semgrep (SAST) – Manual Verification (5 Findings)

No.	File and Finding	Classification	Justification
1	lib/insecurity.ts (Hardcoded RSA Private Key)	True Positive	The unencrypted RSA private block is clearly documented in source code and verified as production application logic via a manual review of the application and not from a test application.
2	routes/redirect.ts (Open Redirect)	True Positive	The "toUrl" value fed to "res.redirect()" is taken directly from user input without validation of the URL being valid.
3	frontend/.../navbar.component.html (Unquoted Attribute)	True Positive	The missing quotes surrounding the {{applicationName}} value allow for the injection of a JavaScript pseudo-protocol. Subtitles controlled by users get injected.
4	routes/videoHandler.ts (XSS via Subtitle)	True Positive	From user-generated subtitle content is injected directly into a <script> context.
5	views/dataErasureForm.hbs (Template Injection)	True Positive	{{useremail}} is rendered insecurely in an HTML attribute with no escaping.

Table 5: OWASP ZAP (DAST) – Manual Verification (5 Findings)

No.	Alert ID & Name	Classification	Justification
1	10096 (Timestamp Disclosure)	False Positive	The flagged timestamp is a standard Unix epoch integer in the frontend JSON state. No sensitive database information was leaked.
2	10017 (Cross-Domain JS Inclusion)	True Positive	Need to check your external scripts for an SRI hash. I started manually checking to confirm that none of them have the integrity attribute.

3	10038 (CSP Header Not Set)	True Positive	When checking the response from HTTP, I did not find any reference to Content-Security-Policy.
4	10110 (Dangerous JS Functions)	True Positive	Angular is still using eval(), as seen by looking at the source bundle file.
5	10063 (Feature-Policy Header)	True Positive	The header is present but uses the deprecated name. It should be called Permissions-Policy.

Table 6: Trivy (SCA) – Manual Verification

No.	File & Finding	Classification	Justification
1	app.guard.spec.ts (JWT)	False Positive	Is included in a unit test specification file only. Never executes in a live environment.
2	last-login-ip.spec.ts (JWT)	False Positive	this is purely a mock token for use when testing an Angular component.
3	lib/insecurity.ts (RSA Key)	True Positive	Critical cryptographic materials hard-coded into the main source of the application show an obvious and sincere breach of secure coding practices.

4.2 Coverage Assessment: Known False Negatives

Tools are unable to identify certain classes of vulnerabilities because we will see below there are at least two false negatives for each tool during this analysis.

Semgrep (SAST) – False Negatives

1. QL Injection Authentication Bypass (CWE-89 / A03:2021). This was verified manually. A crafted payload was submitted using {email = admin@juice-sh.op' --, password = anything} and it was able to log into the application successfully as an admin user and receive an admin JWT. There is no rule in Semgrep for p/security-audit that matches the type of

query construction found on the backend of Juice Shop for that paylaod.

FIGURE 5: Burp/Postman showing SQLi bypass returning admin JWT

2. Business Logic Flaws. Semgrep performs syntactic pattern matching against source code and therefore cannot detect changing the value in a hidden item.price form field will allow an item to be purchased for $0, as it cannot analyze the runtime state or the values of variables at run-time. Performing dynamic analysis or conducting manual penetration testing would be needed to identify such flaws.

OWASP ZAP (DAST) – False Negatives

1. SQL Injection Authentication Bypass (CWE-89). The scan was run with zap-baseline.py, which does not perform authenticated passive scanning, so it cannot submit exploit payloads or accept authentication endpoints. Therefore, the scan missed the bypass entirely.
2. Insecure Direct Object References (IDOR). ZAP was unable to authenticate a session. Therefore it cannot access protected endpoints which would allow it to check that User A cannot view User B's order history. ZAP cannot tell if there is an Insecure Direct Object Reference on the endpoint /api/Orders/123. To detect an Insecure Direct Object Reference, an authenticated spider must be run.

Trivy (SCA) – False Negatives

1. Vulnerable Dependency CVEs. For this test, the fs command was used with Trivy. This command identified secrets and did not fully parse the "package-lock.json" file to check for versions against the NVD, resulting in missing known CVEs for the Juice Shop dependencies, such as older versions of Express or Sequelize.
2. Infrastructure-as-Code Misconfigurations. Trivy was not executed in IaC scanning mode (using "--config"). Therefore, it did not identify potential misconfigurations in the Dockerfile or docker-compose files that may create container breakout vulnerabilities.

5.0 Structured Comparative Matrix

5.1 Vulnerability Detection Accuracy (Dimension 1)

The following matrix compares the quantitative accuracy of each tool based on the manual verification conducted in Section 4.

Dimension	Semgrep (SAST)	OWASP ZAP (DAST)	Trivy (SCA)
Total Findings Detected	7	9	3
True Positives (Verified)	6	7	1
False Positives (Verified)	1	2	2
Known False Negatives (Identified)	SQLi Auth Bypass, Business Logic Flaws	SQLi Auth Bypass, IDOR	Dependency CVEs, IaC Misconfigs

5.2 Vulnerability Coverage & OWASP Top 10 (2024) Mapping (Dimension 2)

OWASP Top 10 (2024) Category	Semgrep Detected?	ZAP Detected?	Trivy Detected?
A03:2024 – Injection (SQLi, XSS)	XSS only	Missed SQLi	No
A02:2024 – Cryptographic Failures	RSA Key exposure	No	RSA Key exposure
A05:2024 – Security Misconfiguration	No	CSP, Headers	No
A01:2024 – Broken Access Control	Open Redirect	Missed IDOR	No
A08:2024 – Software & Data Integrity Failures	No	SRI Missing	Hardcoded secrets

5.3 Ease of Use and Setup (Dimension 3)

Dimension	Semgrep (SAST)	OWASP ZAP (DAST)	Trivy (SCA)
Installation	Moderate (requires pipx/pip).	Simple (Docker pull works).	Trivial (single APT/brew command).
Execution Complexity	High dependency on external registry.	Complex CLI parameter parsing, but baseline script abstracts it well.	Extremely simple (trivy fs .).
Documentation Quality	Excellent (extensive public rulesets).	Excellent (OWASP maintains detailed guides).	Good (clear, well-structured docs).
Learning Curve	Steep (requires understanding of AST).	Moderate (GUI helps, but automation is hard).	Shallow (run and interpret).

6.0 Reflection

The implementation of a mixed-tool vulnerability assessment process using SAST, DAST, and SCA methodologies represented a true transformation from viable and theoretical SSDLC to today's engineering realities. My experience with the project was heavily analytical and relied less on simply clicking through the tools.

The first and most important thing that I learned was that densely containerized security testing absolutely depends on well-defined environment setup. The first error I encountered was a networking allocation error within the target environment due to an unallocated local port on port 3000, and I had to adjust how the external host mapped to the container to define the mapping via port 8080 instead. The importance of setting up dynamically deployable environment configurations in localised sandboxes was thus reinforced, as was the need to always check for port conflicts before starting a vulnerability assessment.

One significant engineering lesson learned was that Semgrep was unable to reach the remote registry/server profile and returned an HTTP 404 error code as a result. In a production environment relying on external network registries, failure to connect to those registries can break CI/CD pipelines. I learned that by using alternate local audit rule sets, it is necessary to maintain offline or fallback definitions of vulnerabilities rather than relying 100% on dynamic web registries.

Finally, through validating the results of Trivy against the raw results, I was able to highlight the ongoing issue of "tool noise." Trivy successfully identified an extremely critical vulnerability with an exposed RSA private key file, but it also resulted in many low-risk testing variables being identified in local spec code blocks thus these variables would be considered false positives in real-world applications. Additionally, I discovered that running ZAP on baseline mode was not effective at detecting injection flaws. The use of active scanning with authenticated context is critical when performing in-depth testing of any vulnerability. My work demonstrated that automated tools are only an amplifier. The ultimate line of defence will require human validation as well as proper scoping and contextual security engineering when implementing automated tools for security testing.

7.0 Appendix

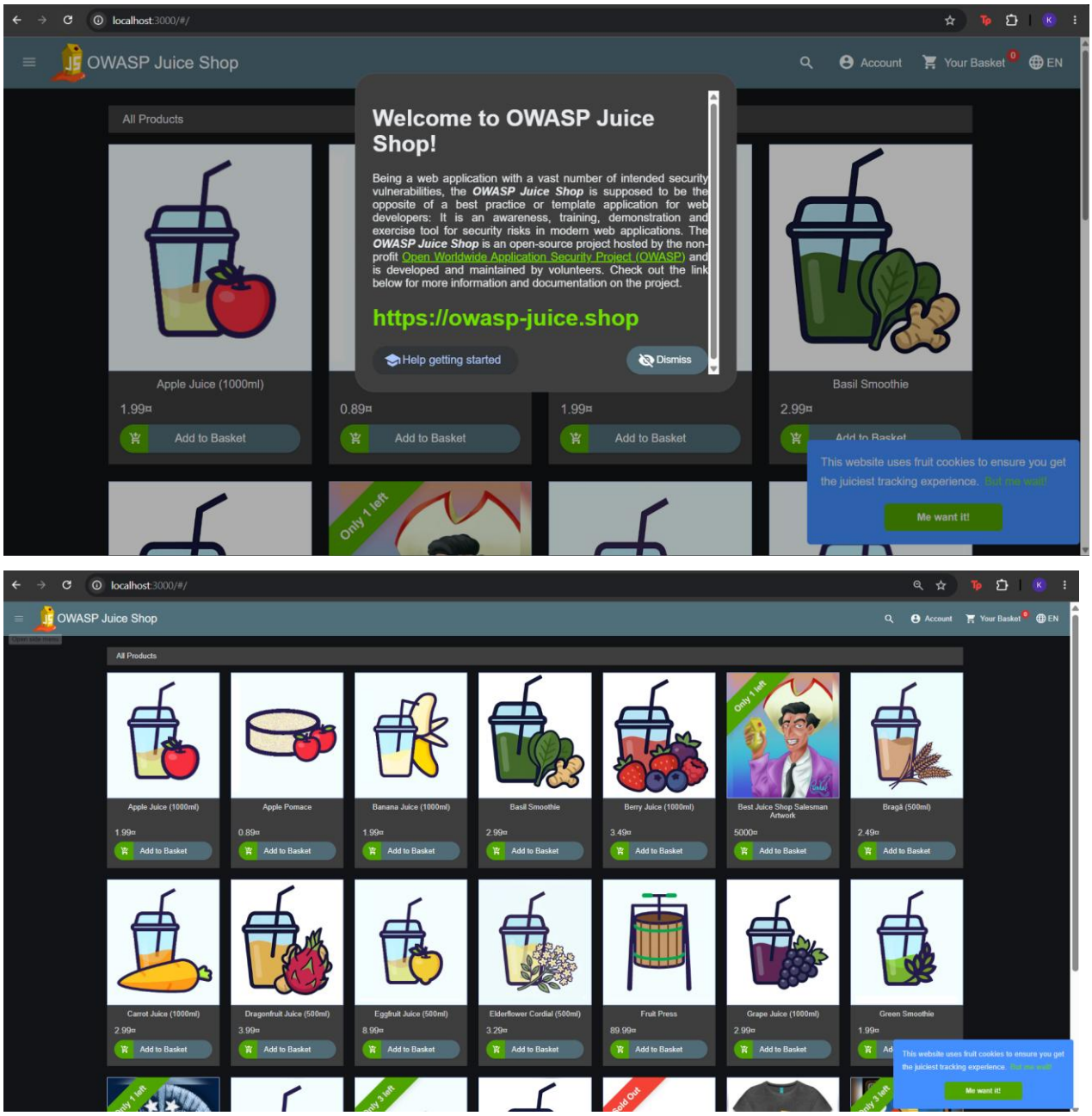

Figure 6: OWASP Juice shop output

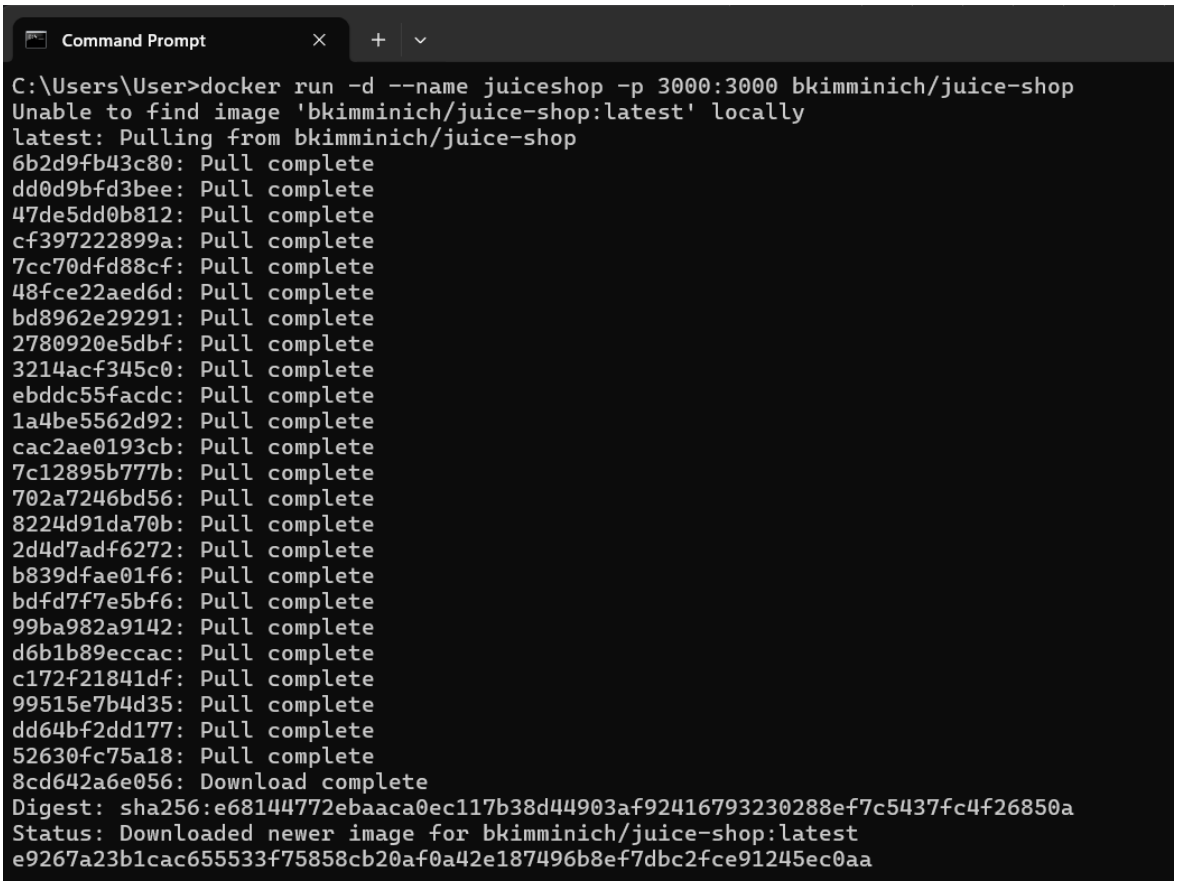

FIGURE 7: Docker run command

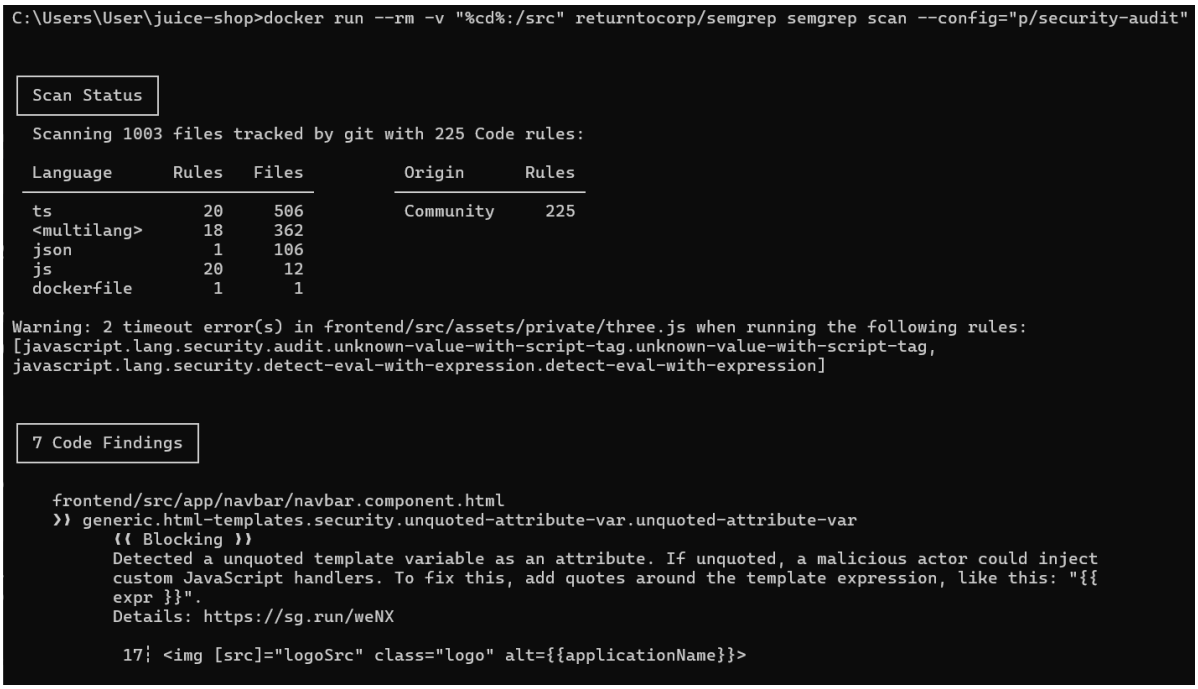

FIGURE 8: Semgrep execution

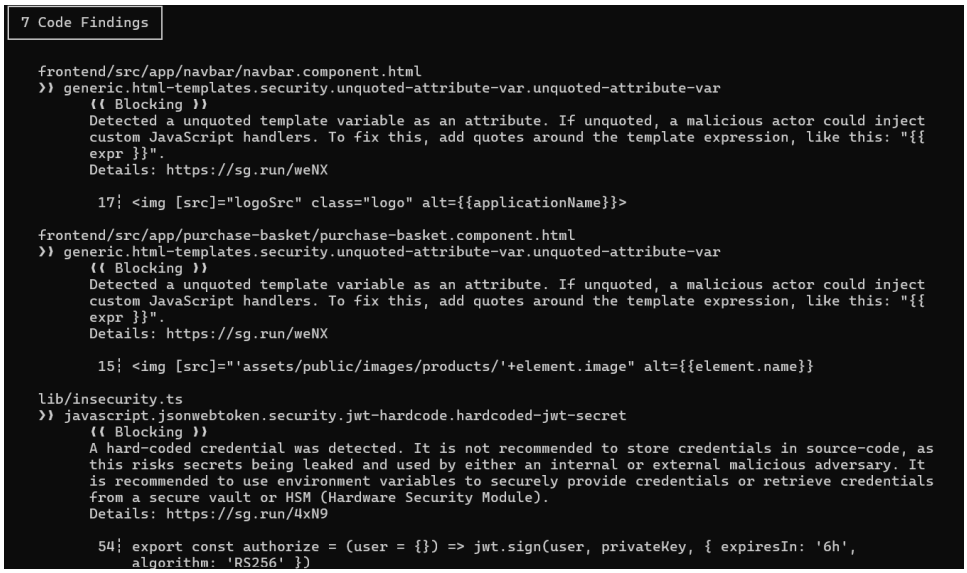

FIGURE 9: Semgrep raw output

```
C:\Users\User\juice-shop>docker run --rm ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t http://host
.docker.internal:8080/ -s
Using the Automation Framework
WARN-NEW: Cross-Domain JavaScript Source File Inclusion [10017] x 5
WARN-NEW: Content Security Policy (CSP) Header Not Set [10038] x 5
WARN-NEW: Non-Storable Content [10049] x 10
WARN-NEW: Deprecated Feature Policy Header Set [10063] x 5
WARN-NEW: Timestamp Disclosure - Unix [10096] x 1
WARN-NEW: Cross-Domain Misconfiguration [10098] x 5
WARN-NEW: Modern Web Application [10109] x 5
WARN-NEW: Dangerous JS Functions [10110] x 2
WARN-NEW: Cross-Origin-Embedder-Policy Header Missing or Invalid [90004] x 10
FAIL-NEW: 0     FAIL-INPROG: 0  WARN-NEW: 9     WARN-INPROG: 0  INFO: 0 IGNORE: 0       PASS: 58

What's next:
    Debug this container error with Gordon → docker ai "help me fix this container error"
```

FIGURE 10: ZAP baseline scan and report

```
C:\Users\User\juice-shop>docker run --rm -v "%cd%:/apps" aquasec/trivy fs /apps
Unable to find image 'aquasec/trivy:latest' locally
latest: Pulling from aquasec/trivy
7003a2b1b982: Pull complete
55afa1ecc21d: Pull complete
47f2314812cd: Pull complete
f47b905a4150: Pull complete
c2584bd48779: Download complete
0b757be1d697: Download complete
Digest: sha256:cffe3f5161a47a6823fbd23d985795b3ed72a4c806da4c4df16266c02accdd6f
Status: Downloaded newer image for aquasec/trivy:latest
2026-07-03T08:46:54Z    INFO    [vulndb] Need to update DB
2026-07-03T08:46:54Z    INFO    [vulndb] Downloading vulnerability DB...
2026-07-03T08:46:54Z    INFO    [vulndb] Downloading artifact...        repo="mirror.gcr.io/aquasec/trivy-db:2"
815.25 KiB / 98.90 MiB [>_____] 0.81% ? p/s ?1.66 MiB / 98.90 MiB [->_____] 2.02% ? p/s ?3.39 MiB / 98.90 MiB [->_____] 3.43% 4.32 MiB p/s ETA 22s4.05 MiB / 98.90 MiB [-->_____] 4.58% 4.32 MiB p/s ETA 21s4.88 MiB / 98.90 MiB [-->_____] 4.94% 4.21 MiB p/s ETA 22s5.27 MiB / 98.90 MiB [-->_____] 5.33% 4.21 MiB p/s ETA 22s5.55 MiB / 98.90 MiB [-->_____] 5.61% 4.21 MiB p/s ETA 22s5.74 MiB / 98.90 MiB [-->_____] 5.80% 4.03 MiB p/s ETA 23s5.90 MiB / 98.90 MiB [-->_____] 5.96% 4.03 MiB p/s ETA 23s6.03 MiB / 98.90 MiB [--->_____] 6.10% 4.03 MiB p/s ETA 23s6.11 MiB / 98.90 MiB [--->_____] 6.30% 3.81 MiB p/s ETA 24s6.34 MiB / 98.90 MiB [--->_____] 6.41% 3.81 MiB p/s ETA 24s6.45 MiB / 98.90 MiB [--->_____] 6.77% 3.60 MiB p/s ETA 25s7.04 MiB / 98.90 MiB [--->_____] 7.12% 3.60 MiB p/s ETA 25s7.35 MiB / 98.90 MiB [--->_____] 7.79% 3.46 MiB p/s ETA 26s8.41 MiB / 98.90 MiB [---->_____] 8.50% 3.46 MiB p/s ETA 26s9.45 MiB / 98.90 MiB [---->_____] 9.56% 3.46 MiB p/s ETA 25s10.22 MiB / 98.90 MiB [----->_____] 10.33% 3.46 MiB p/s ETA 25s11.09 MiB / 98.90 MiB [----->_____] 11.67% 3.47 MiB p/s ETA 25s11.84 MiB / 98.90 MiB [----->_____] 11.97% 3.47 MiB p/s ETA 25s12.39 MiB / 98.90 MiB [------>_____] 13.20% 3.40 MiB p/s ETA 25s13.40 MiB / 98.90 MiB [------>_____] 13.55% 3.40 MiB p/s ETA 25s13.81 MiB / 98.90 MiB [------>_____] 14.38% 3.31 MiB p/s ETA 25s14.68 MiB / 98.90 MiB [------->_____] 14.85% 3.31 MiB p/s ETA 25s15.02 MiB / 98.90 MiB [------->_____] 15.58% 3.22 MiB p/s ETA 25s15.92 MiB / 98.90 MiB [------->_____] 16.10% 3.22 MiB p/s ETA 25s16.59 MiB / 98.90 MiB [-------->_____] 16.78% 3.22 MiB p/s ETA 25s17.19 MiB / 98.90 MiB [-------->_____] 17.38% 3.21 MiB p/s ETA 25s17.76 MiB / 98.90 MiB [-------->_____] 17.95% 3.21 MiB p/s ETA 25s18.20 MiB / 98.90 MiB [-------->_____] 18.41% 3.21 MiB p/s ETA 25s
```

FIGURE 11: Trivy execution

```
epo="mirror.gcr.io/aquasec/trivy-db:2"
2026-07-03T08:47:58Z    INFO    [vuln] Vulnerability scanning is enabled
2026-07-03T08:47:58Z    INFO    [secret] Secret scanning is enabled
2026-07-03T08:47:58Z    INFO    [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2026-07-03T08:47:58Z    INFO    [secret] Please see https://trivy.dev/docs/v0.72/guide/scanner/secret#recommendation for faster secret detection
2026-07-03T08:50:03Z    INFO    Number of language-specific files       num=0

Report Summary
```

Target	Type	Vulnerabilities	Secrets
frontend/src/app/app.guard.spec.ts	text	-	1
frontend/src/app/last-login-ip/last-login-ip.component.spec.ts	text	-	1
lib/insecurity.ts	text	-	1

```
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)


frontend/src/app/app.guard.spec.ts (secrets)
============================================
Total: 1 (UNKNOWN: 0, LOW: 0, MEDIUM: 1, HIGH: 0, CRITICAL: 0)

MEDIUM: JWT (jwt-token)
═══════════════════════
JWT token
─────────────────────
 frontend/src/app/app.guard.spec.ts:46 (offset: 1632 bytes)
─────────────────────
  44           const guard = TestBed.inject(LoginGuard)
  45
  46 [ ocalStorage.setItem('token', '**********************************************************************************************************************
**********************************')
  47           expect(guard.tokenDecode()).toEqual({
─────────────────────
```

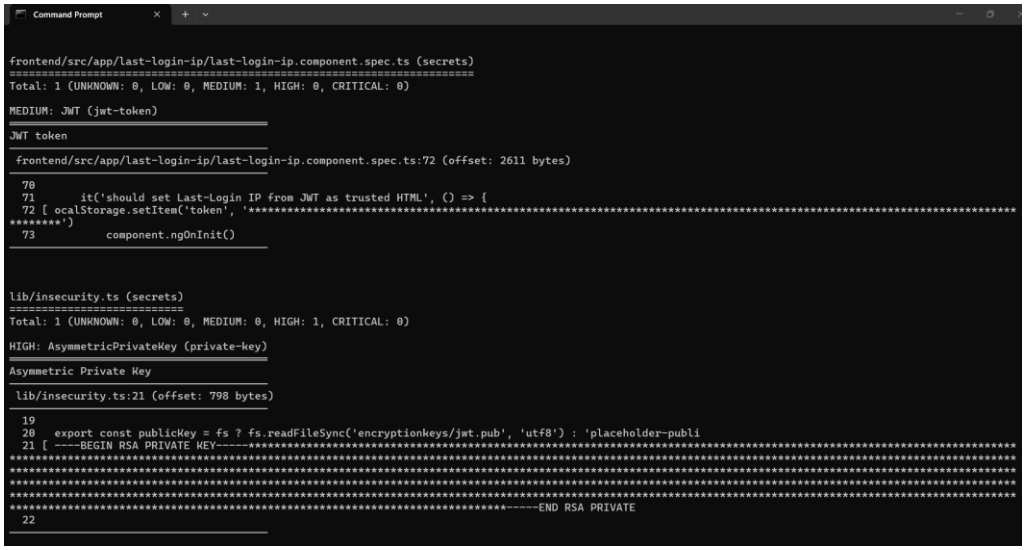

FIGURE 12: Trivy JSON report

```
C:\Users\User\juice-shop>curl http://localhost:8080/rest/basket/1 -H "Authorization:
 Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJzdGF0dXMiOiJzdWNjZXNzIiwiZGF0YSI6eyJ
pZCI6MSwidXNlcm5hbWUiOiIiLCJlbWFpbCI6ImFkbWluQGp1aWNlLXNoLm9wIiwicGFzc3dvcmQiOiIwMTk
yMDIzYTdiYmQ3MzI1MDUxNmYwNjlkZjE4YjUwMCIsInJvbGUiOiJhZG1pbiIsImRlbHV4ZVRva2VuIjoiIiw
ibGFzdExvZ2luSXAiOiIiLCJwcm9maWxlSW1hZ2UiOiJhc3NldHMvcHVibGljL2ltYWdlcy91cGxvYWRzL2R
lZmF1bHRBZG1pbi5wbmciLCJ0b3RwU2VjcmV0IjoiIiwiaXNBY3RpdmUiOnRydWUsImNyZWF0ZWRBdCI6IjI
wMjYtMDctMDMgMDg6MDk6MDUuMzUyICswMDowMCIsInVwZGF0ZWRBdCI6IjIwMjYtMDctMDMgMDg6MDk6MDU
uMzUyICswMDowMCIsImRlbGV0ZWRBdCI6bnVsbH0sImlhdCI6MTc4MzA3MjEzMn0.oS8fsNyjMJQhlvXlGim
-SOSd8Vk8HQoxA_XUo9IqkKTDVFsNhUvyNvIt6Lgpg5w2Dgv0TN3Uq8_2U6fhTnbjLm187lXxr4AwS51DUsJ
H05oZ5qlFOSBsna46Pr2YHzASYIaK7tsKauDc0YgEJGU9GT7qMqvOWPcA5ARcpucMiSA"
{"status":"success","data":{"id":1,"coupon":null,"UserId":1,"createdAt":"2026-07-03T
08:09:07.805Z","updatedAt":"2026-07-03T08:09:07.805Z","Products":[{"id":1,"name":"Ap
ple Juice (1000ml)","description":"The all-time classic.","price":1.99,"deluxePrice"
:0.99,"image":"apple_juice.jpg","createdAt":"2026-07-03T08:09:07.296Z","updatedAt":"
2026-07-03T08:09:07.296Z","deletedAt":null,"BasketItem":{"ProductId":1,"BasketId":1,
"id":1,"quantity":2,"createdAt":"2026-07-03T08:09:07.947Z","updatedAt":"2026-07-03T0
8:09:07.947Z"}},{"id":2,"name":"Orange Juice (1000ml)","description":"Made from oran
ges hand-picked by Uncle Dittmeyer.","price":2.99,"deluxePrice":2.49,"image":"orange
_juice.jpg","createdAt":"2026-07-03T08:09:07.297Z","updatedAt":"2026-07-03T08:09:07.
297Z","deletedAt":null,"BasketItem":{"ProductId":2,"BasketId":1,"id":2,"quantity":3,
"createdAt":"2026-07-03T08:09:07.948Z","updatedAt":"2026-07-03T08:09:07.948Z"}},{"id
":3,"name":"Eggfruit Juice (500ml)","description":"Now with even more exotic flavour
.","price":8.99,"deluxePrice":8.99,"image":"eggfruit_juice.jpg","createdAt":"2026-07
-03T08:09:07.297Z","updatedAt":"2026-07-03T08:09:07.297Z","deletedAt":null,"BasketIt
em":{"ProductId":3,"BasketId":1,"id":3,"quantity":1,"createdAt":"2026-07-03T08:09:07
.948Z","updatedAt":"2026-07-03T08:09:07.948Z"}}]}}
```

FIGURE 13: SQL injection bypass response showing admin JWT